

Software Requirements Specification (SRS)

Billing Brilliance — Multi-Tenant Education Payment Platform

Document Version: 1.0

Date: 2026-05-10

Status: Draft

Prepared By: Engineering Team

Based On: Codebase analysis (frontend + backend), PRD v1.0, 1LINK/1BILL integration specifications

Table of Contents

1. Introduction
 2. Overall Description
 3. System Architecture
 4. User Classes and Characteristics
 5. Functional Requirements
 - 5.1 Authentication and Session Management
 - 5.2 Admin Portal
 - 5.3 School Portal
 - 5.4 ETEA Portal
 - 5.5 Shared/Cross-Portal Features
 6. 1LINK / 1BILL Integration Requirements
 7. Data Requirements
 8. External Interface Requirements
 9. Non-Functional Requirements
 10. Security Requirements
 11. Constraints and Known Limitations
 12. Appendix A — Route Inventory
 13. Appendix B — API Endpoint Inventory
-

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete functional, non-functional, security, and integration requirements for **Billing Brilliance** — a multi-tenant, role-based SaaS platform serving educational institutions and testing authorities in Pakistan for payment collection, billing management, and financial operations. This document is derived from full analysis of the implemented codebase (frontend and backend), product requirements documentation, and 1LINK/1BILL integration standards.

1.2 Scope

Billing Brilliance provides three role-segregated portals operating under a single multi-tenant platform:

Portal	Primary Users	Core Function
Admin Portal	Platform administrators	Tenant, user, biller, transaction, and audit management
School Portal	School administrators, finance staff	Student billing, fee collection, ledger, scholarships, payments
ETEA Portal	ETEA operations teams	Entry test/job posting management, applicant payment orchestration

The platform integrates with the **1LINK/1BILL** inter-bank payment network for real-time bill inquiry, payment processing, and callback handling.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
1BILL	An aggregator service under 1LINK that consolidates biller payment data
1LINK	Inter-bank connectivity network in Pakistan; provides Bill Inquiry and Bill Payment APIs
ETEA	Educational Testing and Evaluation Agency (KPK)
Biller	A registered institution (school, ETEA, or private agency) on the platform
Biller Code	Auto-generated unique identifier for each biller (e.g., <code>1001</code> for schools, <code>2001</code> for ETEA)
Consumer Number	Stable unique payment identifier per student/applicant. Format: <code>{FINTECH_PREFIX}{BILLER_CODE}{PADDED_ENTITY_ID}</code>
Bill ID	Human-readable reference identifier. Format: <code>SCH-GHS-{number}</code> (school) or <code>ETA-MDCAT25-{number}</code> (ETEA)
Fetch Bundle	1LINK endpoint that retrieves a batch of pending bills
BillInquiry	1LINK endpoint to check bill details by consumer number
BillPayment	1LINK endpoint to post a payment against a bill
Tenant	An isolated organizational unit (a school or ETEA entity) on the platform
Callback	POST webhook from 1BILL/1LINK notifying the platform of a completed payment
Idempotency Key	Unique key preventing duplicate processing of the same callback/payment
RBAC	Role-Based Access Control
PKR	Pakistani Rupee (currency used throughout the platform)
CNIC	Computerized National Identity Card (Pakistan national ID)
SaaS	Software as a Service
JWT	JSON Web Token (authentication mechanism)
FSc	Faculty of Science (qualification level in Pakistan)

1.4 References

- `docs/PRD_Billing_Brilliance_1LINK.md` — Product Requirements Document v1.0

- [docs/1Link/1LINK_Generic_REST_Based_Specification_v1_5.md](#) — 1LINK API Specification v1.5
- [docs/1Link/1BILL_Aggregator_Overview.md](#) — 1BILL Aggregator Architecture
- [docs/1Link/1LINK_Data_Network_Guidelines_and_Standards.md](#) — Network Connectivity Standards
- [docs/ETEA_Payment_API.md](#) — ETEA Payment Controller API Contract
- [server/src/index.js](#) — Backend entry point and routing configuration
- [src/App.tsx](#) — Frontend route definitions
- [src/types/index.ts](#) — Core TypeScript type definitions

1.5 Document Overview

This SRS is organized as follows: Section 2 provides overall product context; Section 3 describes the system architecture; Sections 4–6 define user classes and all functional requirements; Sections 7–10 cover data, interfaces, non-functional, and security requirements; Section 11 lists known constraints; Appendices A and B provide complete route and API inventories.

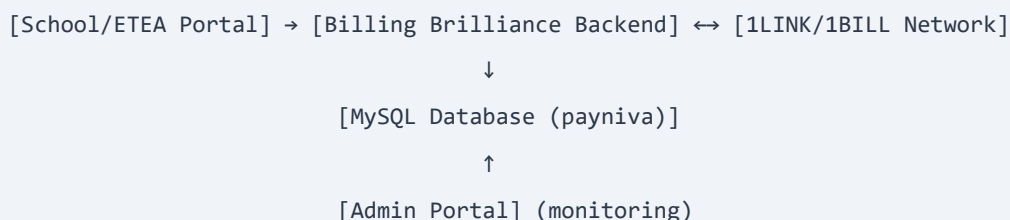
2. Overall Description

2.1 Product Perspective

Billing Brilliance is a greenfield SaaS platform positioned between:

- **Educational institutions** (schools, testing authorities) who need to issue and track fee bills.
- **1LINK/1BILL payment rails** that execute the actual money movement through Pakistan's inter-bank network.
- **Students and applicants** who pay bills through 1LINK-connected channels (ATMs, internet banking, mobile apps).

The platform does **not** execute payments directly — it issues bills with consumer numbers that 1LINK processes, and receives callbacks confirming payment status.



2.2 Product Functions (High-Level)

1. **Multi-tenant onboarding:** Register and configure billers (schools/ETEA) with isolated data scopes.

2. **Student/applicant management:** Create records, generate consumer numbers and bill IDs.
3. **Fee plan management:** Define billing amounts, frequencies, due dates, and late fees.
4. **Invoice lifecycle:** Generate, track, and reconcile invoices (pending → paid / overdue).
5. **Fee ledger:** Per-student double-entry accounting showing charges, discounts, and payments.
6. **1BILL payment flow:** Expose inquiry/payment endpoints, handle callbacks, update ledger.
7. **ETEA payment controller:** Create application-level payment requests, notify ETEA on resolution.
8. **Reporting and analytics:** Dashboards with KPIs, charts, and exportable CSV reports.
9. **Audit trail:** Immutable log of all platform-mutating events.
10. **User and access management:** RBAC with tenant scoping, session timeout, and ban controls.

2.3 Operating Environment

Layer	Technology
Frontend	React 18.3.1, TypeScript, Vite 5.4, TailwindCSS 3.4, shadcn/ui, Radix UI
State Management	Zustand 4.5 (auth), React Query 5.83 (server state)
Charts	Recharts 2.15
Forms	React Hook Form 7.61 + Zod 3.25 (validation)
Backend	Node.js (CommonJS), Express.js
Database	MySQL (via <code>mysql2/promise</code> connection pool; database name: <code>payniva</code>)
Auth	JWT (access: 24h) + Refresh tokens (7d)
Security Middleware	Helmet, CORS, HPP (HTTP Parameter Pollution), express-rate-limit
Logging	Morgan (HTTP) + Winston-style structured logger
Testing (Frontend)	Vitest + jsdom + Testing Library
E2E Testing	Playwright
Build/Bundling	Vite (frontend), Node.js native (backend)

2.4 Design Constraints

- The platform must comply with **1LINK Data Network Guidelines and Standards** for connectivity and protocol requirements.
- All external API traffic must use **HTTPS only**.

- The backend must trust exactly **one reverse-proxy hop** (nginx) in production.
- The frontend is a **Single Page Application (SPA)** — all auth state is held in-memory (Zustand); the 404 "Return Home" link must not cause full-page navigation that loses auth state.
- Consumer numbers must conform to the 1LINK maximum length constraint.

3. System Architecture

3.1 Frontend Architecture

```
src/
├─ App.tsx                # Root router with role-based nested routes
├─ pages/
│   ├─ LoginPage.tsx      # Public login page
│   ├─ admin/             # Admin portal pages
│   ├─ school/            # School portal pages
│   └─ etea/              # ETEA portal pages
├─ components/
│   ├─ DashboardLayout.tsx # Shell: sidebar, header, session guard
│   ├─ GlobalSearch.tsx   # Command-palette search (Ctrl+K)
│   ├─ NotificationCenter.tsx
│   ├─ FilterBar.tsx
│   ├─ TablePagination.tsx
│   ├─ ExportButton.tsx
│   ├─ StatusBadge.tsx
│   ├─ StatCard.tsx
│   └─ ui/                # shadcn/ui primitives
├─ store/
│   ├─ authStore.ts       # Zustand: user + login/logout
│   ├─ paymentStore.ts
│   ├─ eteaSecurityStore.ts
│   └─ orgSecurityStore.ts
├─ services/              # Frontend service layer (API clients)
├─ hooks/                 # Custom React hooks (session timeout, dark mode, etc.)
├─ types/index.ts         # Shared TypeScript types
└─ data/mockData.ts       # Development mock data
```

Routing model: React Router v6 with nested routes. All portal routes are children of `DashboardLayout`, which enforces authentication via Zustand `user` check and redirects unauthenticated users to `/login`.

Dark mode: Managed by `next-themes` with `useDarkMode` hook; persisted in `localStorage`.

3.2 Backend Architecture

```
server/src/
├─ index.js           # Express app setup, middleware, route mounting
├─ config/
│   ├─ index.js       # Env-based configuration (DB, JWT, rate limits, etc.)
│   ├─ database.js    # MySQL connection pool
│   └─ logger.js      # Structured logger
├─ controllers/       # Business logic per domain
├─ routes/            # Express router definitions
├─ middleware/        # Auth, error handling, rate limiting
├─ services/          # Domain services and external integrations
└─ db/               # SQL schema and seed files
```

API base path: `/api/`

Rate limiting: 100 requests/15 min (general); 20 requests/15 min (auth endpoints); 200 requests/1 min (1LINK gateway endpoints).

3.3 Data Architecture

- **Database:** MySQL (`payniva`) via a `mysql2/promise` connection pool (max 20 connections).
- **Timezone:** All connections operate at UTC (`+00:00`).
- **Tenant isolation:** Every data record carries a tenant key; all queries enforce row-level tenant scoping.
- **Audit immutability:** Payment and audit records are append-only.

3.4 External Integration Points

Integration	Direction	Protocol	Purpose
1LINK BillInquiry	Outbound	HTTPS POST	Query bill status by consumer number
1LINK BillPayment	Outbound	HTTPS POST	Post payment against a bill
1LINK Fetch Bundle	Outbound	HTTPS POST	Retrieve batch of pending bills
1BILL Callback	Inbound	HTTPS POST <code>/api/payment/callback</code>	Payment status notification from 1BILL
ETEA Notification	Outbound	HTTPS POST (configurable URL)	Payment status push to ETEA system
ETEA Health	Inbound	HTTPS GET <code>/api/health</code>	Monitoring endpoint

4. User Classes and Characteristics

4.1 Platform Administrator (Admin)

- **Access level:** Full platform visibility across all tenants.
- **Technical proficiency:** Moderate to high; manages billers, users, and monitors transactions.
- **Frequency:** Daily operational use.
- **Portal:** `/admin/*`

4.2 School Administrator

- **Access level:** Scoped to their registered school tenant (`schoolRef`).
- **Sub-roles:** `admin` , `finance` , `staff` , `viewer` (school-level RBAC).
- **Technical proficiency:** Low to moderate; primary focus is student billing and payments.
- **Frequency:** Daily; intensified during fee collection periods.
- **Portal:** `/school/*`

4.3 ETEA Operations User

- **Access level:** Scoped to their ETEA tenant.
- **Technical proficiency:** Moderate; manages postings and payment orchestration.
- **Frequency:** Periodic — intensive during application/exam cycles.
- **Portal:** `/eta/*` (frontend path; internally also referred to as `etea`)

4.4 Indirect Users (Payers)

- Students and applicants who pay bills through 1LINK-connected channels (banks, ATMs, mobile apps).
- They do **not** interact with Billing Brilliance directly; they interact via their banking channel.

4.5 External Systems

- **1BILL/1LINK gateway**: Automated; interacts via REST API.
- **ETEA upstream source system**: Automated; sends payment requests and receives status notifications.

5. Functional Requirements

Notation: Requirements are identified as `FR-[MODULE]-[NUMBER]`. Each requirement has an **Implemented** status indicating whether the feature exists in the current codebase ( `Implemented`,  `Partial`,  `Not Yet`).

5.1 Authentication and Session Management

5.1.1 Login

FR-AUTH-001 — The system shall authenticate users by verifying email, password, and role against the user store.

Implemented:  Frontend mock login (`authStore.ts`);  Backend `/api/auth/login` endpoint.

FR-AUTH-002 — On successful login, the system shall issue a JWT access token and a refresh token containing `userId`, `role`, `tenantId`, and `schoolRef` claims.

Implemented:  Backend (`authController.js`).

FR-AUTH-003 — The login endpoint (`/api/auth/login`) shall accept `email`, `password`, and `role` fields. Requests missing any of these shall receive a 400 response.

Implemented:  (documented in `frontend-gotchas.md`).

FR-AUTH-004 — Access tokens shall expire after 24 hours; refresh tokens shall expire after 7 days.

Implemented:  Configurable via `JWT_EXPIRES_IN` and `JWT_REFRESH_EXPIRES_IN` environment variables.

FR-AUTH-005 — The auth endpoint shall enforce a stricter rate limit: maximum 20 requests per 15-minute window per IP.

Implemented:  (`server/src/index.js`).

5.1.2 Role-Based Route Guards

FR-AUTH-006 — All portal routes (`/admin/*`, `/school/*`, `/eta/*`) shall redirect unauthenticated users to `/login`.

Implemented:  `DashboardLayout.tsx` checks Zustand `user` and navigates to `/login` if null.

FR-AUTH-007 — Backend API endpoints shall validate the JWT and deny requests where the token's `role` or `tenantId` does not match the resource scope.

Implemented:  Middleware in `server/src/middleware/`.

FR-AUTH-008 — School sub-users shall be scoped to their `schoolRef`; all school data reads and writes shall filter by `schoolRef`.

Implemented:  Backend;  Frontend pages must call `mockApi.fetchSchoolUsers(schoolRef)`.

5.1.3 Session Management

FR-AUTH-009 — The system shall display a warning dialog when a user has been inactive for a configurable duration (default: warn at 5 minutes remaining before auto-logout).

Implemented:  `useSessionTimeout.ts` hook integrated into `DashboardLayout.tsx`.

FR-AUTH-010 — The system shall automatically log out the user after the inactivity timeout expires.

Implemented: .

FR-AUTH-011 — User activity (mouse move, keypress, click) shall reset the inactivity timer.

Implemented: .

FR-AUTH-012 — Logout shall clear all auth state and redirect to `/login` without a full page reload that clears SPA state unexpectedly.

Implemented:  Sign out via Zustand reset + React Router navigation.

5.2 Admin Portal

5.2.1 Admin Dashboard (`/admin`)

FR-ADM-001 — The admin dashboard shall display the following platform-wide KPIs:

- Total Revenue (PKR)
- Total Payments (count)
- Pending Payments (PKR)
- Overdue Amount (PKR)
- Active Billers (count)
- Banned Users (count)

Implemented:  (`AdminDashboard.tsx` — currently using mock data).

FR-ADM-002 — The admin dashboard shall render the following charts:

- Revenue trend over time (area chart)
- Payment success rate (bar chart)
- Transaction volume (line chart)

Implemented:  (Recharts, mock data).

FR-ADM-003 — The admin dashboard shall display a recent transactions table showing the last N completed transactions.

Implemented: .

5.2.2 Biller Management (`/admin/billers`)

FR-ADM-004 — Admin shall be able to create a new biller with the following fields:

- Name (required)
- Type: `school` | `org` | `private_agency` (required)
- Email (required, validated)
- Phone (required)

Implemented:  Dialog form in `BillerManagement.tsx`.

FR-ADM-005 — The system shall auto-generate a unique biller code upon creation.

Implemented: .

FR-ADM-006 — Admin shall be able to view all billers and filter by status (`active`, `suspended`, `banned`) and search by name.

Implemented: .

FR-ADM-007 — Admin shall be able to update a biller's status (`active`, `suspended`, `banned`).

Implemented: .

FR-ADM-008 — Biller records shall include: ID, name, type, biller code, email, phone, status, creation date, and optionally an API key.

Implemented:  (defined in `types/index.ts`).

5.2.3 User Management (`/admin/users`)

FR-ADM-009 — Admin shall be able to create a new user with: name, email, role (`admin`, `school`, `org`), and optional tenant reference.

Implemented: .

FR-ADM-010 — Admin shall be able to ban and unban users.

Implemented: .

FR-ADM-011 — Admin shall be able to bulk-import users from a CSV file.

Implemented:  (simulated in frontend; backend endpoint required).

FR-ADM-012 — Admin shall be able to download a CSV template for bulk user import.

Implemented:  (`ExportButton.tsx`).

FR-ADM-013 — The user list shall display: name, email, role, and status with search capability.
Implemented: .

5.2.4 Transaction Monitoring (`/admin/transactions`)

FR-ADM-014 — Admin shall be able to view all platform transactions filterable by:

- Status: `completed`, `pending`, `failed`
- Search by transaction ID or consumer number

Implemented:  (`TransactionList.tsx`).

FR-ADM-015 — Transaction records shall include: transaction ID, biller, consumer number, amount (PKR), status, and date.

Implemented:  (`types/index.ts` — `Transaction` interface).

5.2.5 Cash Flow Analysis (`/admin/cashflow`)

FR-ADM-016 — Admin shall access a cash flow analysis view showing inflows and trends.

Implemented:  Route exists; component is a stub.

5.2.6 Reports (`/admin/reports`)

FR-ADM-017 — Admin shall access platform-level reports including:

- Revenue by tenant/biller
- Payment success/failure rates
- Transaction volumes over time

Implemented:  Route exists; component is a stub.

5.2.7 Audit Trail (`/admin/audit`)

FR-ADM-018 — Admin shall be able to view the platform audit log with entries for all mutating events (create/update/delete/payment-critical actions).

Implemented:  (`AuditTrail.tsx` + `auditController.js` + `/api/audit-logs` route).

FR-ADM-019 — Admin shall be able to export the audit log.

Implemented:  (CSV export via `ExportButton`).

FR-ADM-020 — Audit log entries shall include: timestamp, actor (user ID/email), action, resource type, resource ID, and tenant.

Implemented: .

5.2.8 API Health Sandbox

FR-ADM-021 — Admin shall have access to an API sandbox to probe 1LINK inquiry, payment, and fetch-bundle endpoints and view raw request/response payloads.

Implemented:  Referenced in PRD; backend routes exist (`/api/admin-tools`).

5.2.9 Settings (`/admin/settings`)

FR-ADM-022 — Admin shall have a settings page for platform configuration.

Implemented: ❌ Navigation item exists in sidebar but route/component is missing (404).

5.3 School Portal

5.3.1 School Dashboard (`/school`)

FR-SCH-001 — The school dashboard shall display the following KPIs:

- Total Students (count)
- Amount Collected This Month (PKR)
- Outstanding Balance (PKR, aggregate)
- Defaulters Count
- Today's Payments (count)

Implemented: ✅ (`SchoolDashboard.tsx`).

FR-SCH-002 — The school dashboard shall render the following charts:

- Monthly collection vs. target (bar chart)
- Collection by fee head (pie chart)
- Students by class (bar chart)

Implemented: ✅.

FR-SCH-003 — The school dashboard shall display a recent payments list and overdue alerts.

Implemented: ✅.

FR-SCH-004 — Dashboard stat cards for Students and Defaulters shall be clickable, navigating to `/school/students` and `/school/defaulters` respectively.

Implemented: ✅.

5.3.2 Student Management (`/school/students`)

FR-SCH-005 — School users shall be able to add a new student with the following fields:

- Name (required)
- Father Name (required)
- Class: 1–10 (required)
- Section: A, B, or C (required)
- Phone (required, auto-formatted)
- CNIC (required, auto-formatted, unique)
- Gender: male/female (required)
- Date of Birth (required)
- Address (required)
- Bus service toggle with start/end month and monthly fee

Implemented:  Dialog form in `StudentList.tsx`.

FR-SCH-006 — The system shall automatically generate:

- A unique **Consumer Number** using format: `{FINTECH_PREFIX}{BILLER_CODE}{PADDED_STUDENT_ID}` (prefix: `123456`, school biller code: `1001`)
- A **Bill ID** using format: `SCH-GHS-{zero-padded-number}`

Implemented:  `generateConsumerNumber()` in `mockData.ts`; backend equivalent required.

FR-SCH-007 — Consumer numbers shall be stable (not change after assignment) and globally unique across the platform.

Implemented:  (by design).

FR-SCH-008 — School users shall be able to bulk-import students from a CSV file.

Implemented:  (simulated; backend persistence required).

FR-SCH-009 — A CSV template for student import shall be downloadable.

Implemented: .

FR-SCH-010 — School users shall be able to export the student list to CSV.

Implemented:  (`ExportButton.tsx`).

FR-SCH-011 — The student list shall support:

- Text search by name, consumer number, CNIC, or roll number
- Filter by class
- Pagination (default: 25 per page)

Implemented: .

FR-SCH-012 — Student records shall include: name, father name, roll number, class, section, phone, CNIC, consumer number, bill ID, status (`active` / `inactive`), billerId, balance, admission date, gender, date of birth, address, and bus service fields.

Implemented:  (`types/index.ts` — `Student` interface).

FR-SCH-013 — CNIC shall be validated for uniqueness within the school tenant to prevent duplicate student records.

Implemented:  (frontend validation in `ApplicantList.tsx` pattern; apply consistently to `StudentList.tsx`).

5.3.3 Fee Plan Management (`/school/fee-plans`)

FR-SCH-014 — School users shall be able to create fee plans with the following fields:

- Plan name
- Amount (PKR)
- Frequency: monthly/quarterly/annually
- Due date (day of month/quarter)
- Late fee amount (PKR)

- Type: `tuition` | `additional`

Implemented:  (`FeePlans.tsx`). Sample plans: Standard Monthly (Rs15,000), Premium Monthly (Rs25,000), Quarterly (Rs42,000), Annual (Rs150,000).

FR-SCH-015 — School users shall be able to view, edit, and deactivate fee plans.

Implemented:  View implemented; edit/deactivate partial.

5.3.4 Payment Programs (`/school/payment-programs`)

FR-SCH-016 — School users shall be able to assign a fee plan to all students of a given class/section (bulk assignment).

Implemented:  (`PaymentPrograms.tsx`).

FR-SCH-017 — School users shall be able to assign a fee plan to individual students.

Implemented:  .

FR-SCH-018 — The system shall prevent duplicate assignment of the same plan to the same student.

Implemented:  (enforced in assignment logic).

FR-SCH-019 — Payment program assignments shall track: student name, consumer number, class/section, plan name, amount, frequency, status (`active` / `pending` / `completed`), assigned date, and next due date.

Implemented:  (`types/index.ts` — `PaymentPlanAssignment` interface).

FR-SCH-020 — School users shall be able to filter assignments by class, plan, and student name.

Implemented:  .

5.3.5 Fee Ledger (`/school/fee-ledger`)

FR-SCH-021 — School users shall be able to select a student and view their complete fee ledger.

Implemented:  (`FeeLedger.tsx` — student selector for first 30 students).

FR-SCH-022 — The ledger shall display the following per-entry fields: date, description, debit (charges), credit (payments), running balance, and reference (transaction ID).

Implemented:  .

FR-SCH-023 — The ledger shall display summary cards: Total Charged, Total Paid, Outstanding Balance, and entry count.

Implemented:  .

FR-SCH-024 — Ledger entries shall include all applicable charge types:

- Gross tuition fee
- Transport / bus fee
- Scholarship discount (negative debit)
- Late fines

- Payment allocations ("Payment via 1Bill")
- Running balance

Implemented:  (`generateLedger()` in `mockData.ts`).

FR-SCH-025 — The ledger student selector shall display the student's Bill ID, Father Name, and CNIC.

Implemented: .

5.3.6 Scholarships (`/school/scholarships`)

FR-SCH-026 — School users shall be able to create scholarships with:

- Name
- Type: `percentage` | `fixed`
- Value (percentage or PKR amount)
- Start date / end date (or lifetime flag)
- Status: `active` | `expired` | `inactive`

Implemented:  (`Scholarships.tsx` ; types defined in `types/index.ts`).

FR-SCH-027 — School users shall be able to assign scholarships to individual students, classes, or sections.

Implemented:  (`StudentScholarshipAssignment` interface defined).

FR-SCH-028 — Scholarship discounts shall be reflected as negative debit entries in the student's fee ledger.

Implemented:  (ledger generation logic).

5.3.7 Invoices / Billing (`/school/billing` , `/school/invoices`)

FR-SCH-029 — The system shall generate invoices for students based on their active payment program.

Implemented:  Invoice display exists; automated generation backend logic required.

FR-SCH-030 — School users shall be able to view and filter invoices by:

- Status: `pending` | `paid` | `overdue`
- Search by student name or invoice number

Implemented:  (`InvoiceList.tsx`).

FR-SCH-031 — Invoice records shall include: invoice number, student name, consumer number, month, amount (PKR), late fee, status, due date, and biller ID.

Implemented:  (`types/index.ts` — `Invoice` interface).

FR-SCH-032 — School users shall be able to configure billing policy: automatic, manual, or hybrid invoice generation.

Implemented:  Referenced in PRD; not yet implemented in frontend or backend.

5.3.8 Defaulters (/school/defaulters)

FR-SCH-033 — The defaulters page shall display all students with a positive outstanding balance.

Implemented: ✓ (filtered from `students` where `balance > 0`).

FR-SCH-034 — The defaulters view shall show: student name, father name, class, CNIC, phone, and amount due.

Implemented: ✓ (`Defaulters.tsx`).

FR-SCH-035 — The view shall display aggregate stats: total defaulters count and total outstanding amount.

Implemented: ✓.

FR-SCH-036 — School users shall be able to search defaulters by name or CNIC and paginate results.

Implemented: ✓.

FR-SCH-037 — School users shall be able to export the defaulters list to CSV.

Implemented: ✓.

FR-SCH-038 — School users shall be able to trigger SMS payment reminders for defaulters (single and bulk).

Implemented: ⚠ Button exists in UI; SMS integration not implemented.

FR-SCH-039 — The system shall classify students into risk tiers: `current`, `watch`, `high-risk`, `critical`.

Implemented: ✓ (`StudentRiskTier` type and `StudentFinancialSnapshot` interface in `types/index.ts`).

5.3.9 Payments (/school/payments)

FR-SCH-040 — School users shall be able to perform bill inquiries by consumer number via the 1BILL-compatible flow.

Implemented: ⚠ UI exists; requires backend 1LINK integration for production.

FR-SCH-041 — School users shall be able to post a manual payment against a bill.

Implemented: ⚠ Partial stub.

FR-SCH-042 — The payments page shall display a realtime payment feed showing recent payments and daily totals.

Implemented: ⚠ Referenced in PRD; partial in frontend.

FR-SCH-043 — Successful payment posting shall trigger reconciliation: update invoice status, create ledger entry, create transaction record, and emit audit event.

Implemented: ⚠ Backend `billingController.js` and `paymentController.js` exist; full reconciliation chain requires verification.

5.3.10 School Reports (/school/reports)

FR-SCH-044 — School users shall access school-level reports including:

- Monthly collection vs. target
- Collection by fee head
- Class-level analytics
- Risk tier distribution

Implemented: ⚠️ Route exists; `SchoolReports.tsx` is a stub.

5.3.11 School Settings (`/school/settings`)

FR-SCH-045 — School users shall access a settings page for school configuration (billing policy, notification preferences, sub-user management).

Implemented: ❌ Navigation item exists in sidebar; route/component missing (404).

5.3.12 School Sub-User Management

FR-SCH-046 — School administrators shall be able to create and manage school sub-users with roles: `admin`, `finance`, `staff`, `viewer`.

Implemented: ✅ Backend scoped to `schoolRef`; frontend references `mockApi.fetchSchoolUsers(schoolRef)`.

5.4 ETEA Portal

Note: The portal is referred to as **ETEA** throughout the codebase (path: `/eta/*`). Frontend files are named with the `etea` prefix. The backend route path `/api/etea` and `/api/applicants` both serve ETEA operations.

5.4.1 ETEA Dashboard (`/eta`)

FR-ETEA-001 — The ETEA dashboard shall display the following KPIs:

- Active Postings (count)
- Total Applicants (count)
- Fee Collected Today (PKR)
- Pending Payments (count)
- Admit Cards Issued (count)

Implemented: ✅ (`ETADashboard.tsx`).

FR-ETEA-002 — The ETEA dashboard shall render:

- Applications per posting (horizontal bar chart)
- Applicant pipeline progress (progress bars per stage)
- Collection trend over time (area chart)

Implemented: .

FR-ETEA-003 — The applicant pipeline shall track the following stages in order: Submitted → Fee Pending → Fee Paid → Roll Assigned → Test Scheduled → Appeared → Result Pending → Selected → Rejected

Implemented: .

FR-ETEA-004 — Dashboard stat cards for Active Postings and Total Applicants shall navigate to `/eta/postings` and `/eta/applicants` respectively.

Implemented: .

5.4.2 Postings Management (`/eta/postings`)

FR-ETEA-005 — ETEA users shall be able to create, view, and manage job/test postings with:

- Title
- Type: `entry_test` | `job`
- Department/organization
- Total seats
- Application fee (PKR)
- Application deadline
- Status: `active` | `closed` | `draft`

Implemented:  (`ETEAPostings.tsx`). Sample: MDCAT 2025 (Rs3,500, 5,000 seats), ECAT 2025 (Rs3,500, 3,000 seats), Lecturer Physics KPK (Rs2,500, 150 seats), SST Math KPK (Rs2,000, 300 seats, draft).

FR-ETEA-006 — Postings shall track application counts and seat utilization.

Implemented: .

5.4.3 Applicant Management (`/eta/applicants`)

FR-ETEA-007 — ETEA users shall be able to add applicants with:

- Name, Father Name (required)
- CNIC (required, unique, auto-formatted)
- Phone, Email
- District (domicile)
- Gender, Date of Birth
- Qualification: FSc Pre-Med / Pre-Eng / BA / BSc / MA / MSc
- Posting ID (linked to a posting)

Implemented:  (`ApplicantList.tsx` with CNIC duplicate validation).

FR-ETEA-008 — The system shall auto-generate:

- **Consumer Number:** `{FINTECH_PREFIX}2001{PADDED_APPLICANT_ID}` (ETEA biller code: `2001`)

- **Bill ID:** `ETA-MDCAT25-{zero-padded-number}`

Implemented: .

FR-ETEA-009 — Applicant records shall include: name, father name, CNIC, phone, email, district, gender, DOB, qualification, consumer number, bill ID, payment status, application status, posting ID, roll number (if assigned), test center (if assigned), marks (if appeared), and applied date.

Implemented: .

FR-ETEA-010 — The applicant list shall support:

- Search by name, CNIC, or roll number
- Filter by application status or posting
- Pagination (default: 25 per page)
- Export to CSV

Implemented: .

5.4.4 ETEA Payment Controller

The ETEA payment controller treats the **application** as the payment unit. The ETEA system submits payment creation requests; Billing Brilliance manages bill lifecycle and notifies ETEA of resolution.

FR-ETEA-011 — The system shall expose an endpoint to create an ETEA payment request:

- Input: `applicant_id`, `application_id`, `posting_id`, `amount`, `due_date`, `callback_url`
- Output: generated `bill_id`, `consumer_number`, and initial status `pending`
- Exactly one payment record per `application_id` (deduplicated)

Implemented:  Backend `eteaPaymentController.js` + route `/api/etea` or `/api/applicants`.

FR-ETEA-012 — The system shall provide a status lookup endpoint by `application_id` returning current payment state.

Implemented: .

FR-ETEA-013 — Upon receiving a confirmed 1BILL callback for an ETEA payment, the system shall:

1. Validate callback signature and idempotency
2. Update payment record status to `paid` (or `failed`)
3. Record `paid_at`, `transaction_id`
4. POST notification to the configured ETEA endpoint (`/api/etea/payment-status`) with status payload

Implemented:  Backend (`eteaPaymentController.js`, callback handler).

FR-ETEA-014 — ETEA payment records shall include all fields specified in Section 7.2.

Implemented: .

FR-ETEA-015 — The system shall expose a health endpoint (`GET /api/health`) returning service status for monitoring.

Implemented: .

5.4.5 ETEA Invoices (`/eta/invoices`)

FR-ETEA-016 — ETEA users shall be able to view applicant invoices with: invoice number, applicant name, amount, status, and due date.

Implemented:  (`ETAIInvoices.tsx` ; currently reuses school invoice data — ETEA-specific data binding required).

5.4.6 ETEA Payments (`/eta/payments`)

FR-ETEA-017 — ETEA users shall access a payment management view showing payment records, realtime paid feed, and daily totals.

Implemented:  Stub (`ETAPayments.tsx`).

5.4.7 ETEA Reports (`/eta/reports`)

FR-ETEA-018 — ETEA users shall access posting-level reports: payment request count, paid ratio, collection totals, and verified transaction counts.

Implemented:  Stub (`ETAREports.tsx`).

5.4.8 ETEA Modules Awaiting Routing (Post-MVP)

The following pages exist in source but are **not wired** in `App.tsx` routing. They are planned for Release 2:

Module	Path	Status
Roll Assignment	<code>/eta/roll-assignment</code>	 No route — 404 if navigated
Admit Cards	<code>/eta/admit-cards</code>	 No route — 404 if navigated
Results	<code>/eta/results</code>	 No route — 404 if navigated
Service List	<code>/eta/services</code>	 No route — 404 if navigated
ETEA Payment Programs	<code>/eta/payment-programs</code>	 No route — 404 if navigated
ETEA Settings	<code>/eta/settings</code>	 No route — 404 if navigated

FR-ETEA-019 — The sidebar navigation shall not display links to unimplemented routes, OR those routes shall be implemented, to prevent 404 dead ends.

Implemented:  6 broken navigation items currently present.

5.5 Shared / Cross-Portal Features

5.5.1 Dashboard Layout (`DashboardLayout.tsx`)

FR-UX-001 — All authenticated portal pages shall be rendered inside a shared shell providing:

- Fixed sidebar with role-specific navigation groups (Overview, Management, Finance, Analytics, System)
- Collapsible/hamburger sidebar on mobile
- Header with breadcrumb, global search, notification bell, and dark mode toggle
- User profile card in sidebar footer with logout button

Implemented: .

5.5.2 Global Search (`GlobalSearch.tsx`)

FR-UX-002 — A global command palette shall be accessible via `Ctrl+K` / `Cmd+K`.

Implemented: .

FR-UX-003 — The global search shall support searching: students (→ `/school/students`), applicants (→ `/eta/applicants`), and transactions (→ `/admin/transactions`).

Implemented: .

FR-UX-004 — Results shall appear for queries longer than 1 character; selecting a result navigates to the relevant page and closes the palette.

Implemented: .

5.5.3 Notification Center (`NotificationCenter.tsx`)

FR-UX-005 — A notification center accessible via a bell icon in the header shall display platform notifications with unread count badge.

Implemented: .

FR-UX-006 — Notification types shall include: `payment` (green), `applicant` (blue), and `alert` (orange).

Implemented: .

FR-UX-007 — Users shall be able to mark individual notifications as read, mark all as read, and clear all notifications.

Implemented: .

5.5.4 Dark Mode

FR-UX-008 — The platform shall support light and dark mode toggle, persisted in `localStorage`.

Implemented:  (`next-themes`, `useDarkMode.ts`).

5.5.5 Export Functionality

FR-UX-009 — Key data tables (students, applicants, defaulters, audit logs) shall have an export dropdown offering:

- Export to CSV (download file)
- Print View (`window.print()`)

Implemented:  (`ExportButton.tsx`).

FR-UX-010 — CSV export shall produce properly quoted, header-labelled files.

Implemented: .

5.5.6 Filtering and Pagination

FR-UX-011 — All list pages shall provide a `FilterBar` with text search and dropdown filters relevant to the domain.

Implemented:  (`FilterBar.tsx`).

FR-UX-012 — All list pages shall support pagination with configurable page sizes: 10, 25, 50, 100.

Implemented:  (`TablePagination.tsx`).

5.5.7 Status Display

FR-UX-013 — All status values shall be rendered as colored badges using the following semantic mapping:

Status Value	Color	Semantic
<code>active</code> , <code>paid</code> , <code>completed</code>	Green	Success
<code>pending</code> , <code>partial</code> , <code>suspended</code>	Orange	Warning
<code>overdue</code> , <code>failed</code> , <code>banned</code>	Red	Destructive
<code>expired</code> , <code>inactive</code>	Gray	Muted

Implemented:  (`StatusBadge.tsx`).

5.5.8 Audit Events

FR-OPS-001 — All create, update, delete, payment, and access-critical operations shall emit an auditable event to the audit log.

Implemented:  Backend `auditController.js` + `/api/audit-logs` route.

6. 1LINK / 1BILL Integration Requirements

6.1 Transaction Types

The platform shall support the following 1LINK transactions:

Transaction	Endpoint	Direction
Fetch Bundle	POST /v1/Transaction/Fetchbundle	Outbound to 1LINK
Bill Inquiry	POST /api/1.0/Payments/BillInquiry	Outbound to 1LINK
Bill Payment	POST /api/1.0/Payments/BillPayment	Outbound to 1LINK

Backend routes: /api/1link/*, /api/fetchbundle, /api/bundles

6.2 Inquiry Requirements

FR-1LINK-001 — Bill Inquiry requests shall include: username, password, consumer_number, bank_mnemonic, and reserved fields. All fields shall honor 1LINK fixed-length constraints.

Implemented:  oneLinkController.js.

FR-1LINK-002 — The reserved field shall support packing for: CNIC, account ID, bundle ID, and support info segments.

Implemented: .

FR-1LINK-003 — The system shall parse and map response_Code and bill_status from inquiry responses.

Implemented: .

6.3 Payment Requirements

FR-1LINK-004 — Bill Payment requests shall include a unique transaction identity tuple:

consumer_number + tran_auth_id + tran_date + tran_time.

Implemented: .

FR-1LINK-005 — The system shall detect and handle duplicate payment tuples deterministically (reject with appropriate response code, not double-post).

Implemented: .

FR-1LINK-006 — The system shall persist the gateway response and map it to an internal status.

Implemented: .

6.4 Response Code Mapping

FR-1LINK-007 — The system shall map 1LINK response codes to internal business states:

1LINK Code	Internal State	Meaning
00	completed	Success
01	not_found	Invalid/not found
02	blocked	Blocked/unknown error
03	duplicate	Bad/duplicate transaction
04	invalid_data	Invalid data
05	processing_failed	Processing failure
06	already_paid	Already paid (where applicable)

Implemented: .

6.5 Callback and Notification Requirements

FR-1LINK-008 — The system shall expose a callback endpoint at `POST /api/payment/callback` to receive payment status notifications from 1BILL.

Implemented: .

FR-1LINK-009 — Callback processing shall:

1. Validate webhook signature (`X-Webhook-Signature` header)
2. Check idempotency key (`X-Idempotency-Key` header) to prevent duplicate processing
3. Validate source IP against whitelist
4. Update payment status
5. Emit audit event

Implemented:  `org.webhookSecret` and `requireWebhookSignature` config; IP whitelist middleware.

FR-1LINK-010 — Callback processing shall be exactly-once effective at the business level (idempotent).

Implemented: .

FR-1LINK-011 — For ETEA payments, after callback processing, the system shall POST a payment status notification to the ETEA endpoint pattern `POST /api/etea/payment-status`.

Implemented:  `eteaPaymentController.js`.

6.6 Network Requirements (1LINK Standards)

NR-001 — The integration shall support dual connectivity to 1LINK primary and DR sites.

NR-002 — Routing to 1LINK external edge shall use static routing only.

NR-003 — IPSec VPN configuration shall use: IKEv2, AES-256, SHA-256, DH Group 19, PFS enabled. Aggressive mode shall be disabled.

NR-004 — Wireless last-mile media types prohibited by 1LINK guidelines shall not be used.

6.7 1LINK Credentials Configuration

Config Key	Env Variable	Default (Dev Only)
Username	ONELINK_USERNAME	demo-user
Password	ONELINK_PASSWORD	demo-pass
Bank Mnemonic	ONELINK_BANK_MNEMONIC	MBLINK01

7. Data Requirements

7.1 Core Entities

Entity	Scope	Key Identifiers
Tenant	Platform	id, name, type
User	Platform	id, email, role, tenantId, schoolRef
Biller	Platform	id, billerCode, type
Student	School tenant	id, consumerNumber, billId, billerId
Scholarship	School tenant	id, type, value
StudentScholarshipAssignment	School tenant	studentId, scholarshipId
FeePlan	School tenant	id, frequency, amount
PaymentPlanAssignment	School tenant	studentId, feePlanId
Invoice	School/ETEA tenant	id, invoiceNumber, consumerNumber, status
LedgerEntry	School tenant	studentId, date, debit, credit, balance
Transaction	Platform	id, transactionId, consumerNumber, status
AuditLog	Platform	id, actor, action, resource, tenantId
ETEPaymentRecord	ETEA tenant	See 7.2
ETEPaymentNotification	ETEA tenant	id, paymentId, status, sentAt
Posting	ETEA tenant	id, title, type, seats, fee
Applicant	ETEA tenant	id, cnic, consumerNumber, billId, postingId

7.2 ETEA Payment Record Fields

Field	Type	Description
<code>id</code>	UUID	Primary key
<code>application_id</code>	string	Unique per application (deduplication key)
<code>applicant_id</code>	string	Reference to applicant
<code>posting_id</code>	string	Reference to posting
<code>bill_id</code>	string	Generated bill identifier
<code>amount</code>	decimal	Fee amount in PKR
<code>status</code>	enum	<code>pending</code> <code>paid</code> <code>failed</code> <code>expired</code>
<code>due_date</code>	datetime	Payment deadline
<code>expiry_date</code>	datetime	Bill expiry
<code>created_at</code>	datetime	Record creation timestamp
<code>paid_at</code>	datetime	Payment confirmation timestamp
<code>transaction_id</code>	string	1LINK transaction reference
<code>callback_url</code>	string	ETEA notification endpoint

7.3 Consumer Number Format

```
{FINTECH_PREFIX}{BILLER_CODE}{PADDED_ENTITY_ID}
```

Where:

FINTECH_PREFIX = "123456" (configurable via FINTECH_PREFIX env var)

BILLER_CODE = "1001" (school) | "2001" (ETEA)

PADDED_ENTITY_ID = zero-padded to 14 digits

Example (Student 1, School 1001):

```
1234561001000000000001
```

Example (Applicant 1, ETEA 2001):

```
1234562001000000000001
```

Consumer numbers must comply with 1LINK maximum consumer number length constraints.

7.4 Data Governance

- Every record shall carry a tenant key for row-level enforcement.
- Payment and audit records shall be append-only (immutable after creation).
- CNIC fields shall be masked in UI logs and protected at rest.
- Auth credential values (passwords, secrets) shall never appear in application logs.
- Indexes shall be partitioned by tenant ID and date for transaction tables.

8. External Interface Requirements

8.1 User Interface

UI-001 — The frontend shall be a responsive Single Page Application (SPA) built with React 18 + TypeScript.

UI-002 — The UI shall support light and dark themes.

UI-003 — The UI shall be mobile-responsive; the sidebar shall collapse to a hamburger menu on small viewports.

UI-004 — All forms shall provide inline validation feedback using React Hook Form + Zod schemas.

UI-005 — Phone and CNIC fields shall auto-format on input.

UI-006 — Loading, empty, and error states shall be explicitly handled and displayed using `EmptyState` and `ErrorBoundary` components.

8.2 Backend API Interface

API-001 — All backend endpoints shall be prefixed with `/api/`.

API-002 — The API shall return JSON responses with consistent structure: `{ data, message, error }`.

API-003 — Authentication shall use Bearer JWT tokens in the `Authorization` header.

API-004 — ETEA API-key-protected endpoints shall require the `X-API-Key` header.

API-005 — Requests with a body shall enforce a 100 KB size limit (bulk endpoints may set a higher limit explicitly).

API-006 — All responses shall include appropriate HTTP status codes (200, 201, 400, 401, 403, 404, 409, 429, 500).

API-007 — CORS shall be configured to allow the frontend origin (`CORS_ORIGIN` env var; default: `http://localhost:5173`) with credentials.

API-008 — Custom request headers allowed: `Content-Type`, `Authorization`, `X-API-Key`, `X-Webhook-Signature`, `X-Idempotency-Key`, `X-Tenant-Id`.

8.3 Database Interface

DB-001 — The system shall use MySQL via a `mysql2/promise` connection pool (max 20 connections; configurable).

DB-002 — All timestamps shall be stored in UTC (`+00:00`).

DB-003 — The database name is `payniva` (configurable via `DB_NAME` env var).

DB-004 — The system shall gracefully handle connection failures and log errors without exposing credentials.

8.4 1LINK External Interface

EXT-001 — All 1LINK communication shall use HTTPS only.

EXT-002 — 1LINK credentials (username, password, bank mnemonic) shall be loaded from environment variables and never hardcoded in source.

EXT-003 — The platform shall support dual endpoints (primary + DR) for 1LINK connectivity.

9. Non-Functional Requirements

9.1 Performance

NFR-PERF-001 — P95 API response latency for core read paths shall be ≤ 500 ms under nominal load.

NFR-PERF-002 — P95 callback processing time shall be ≤ 1 second, excluding external dependency latency.

NFR-PERF-003 — The frontend bundle shall load and render the login page within 3 seconds on a standard broadband connection.

9.2 Reliability

NFR-REL-001 — Core payment services shall target 99.9% monthly uptime.

NFR-REL-002 — Callback processing shall guarantee exactly-once business-level effect (idempotent semantics).

NFR-REL-003 — Database connection failures shall be logged and surfaced as 503 responses, not silent data corruption.

9.3 Scalability

NFR-SCALE-001 — The platform shall support multi-tenant growth without cross-tenant data leakage.

NFR-SCALE-002 — Transaction and audit tables shall be indexed by tenant ID and date to support high-volume queries.

NFR-SCALE-003 — The backend connection pool shall be configurable via `DB_CONNECTION_LIMIT` (default: 20).

9.4 Observability

NFR-OBS-001 — All HTTP requests shall be logged with Morgan in `combined` format in production and `dev` format in development.

NFR-OBS-002 — Application events shall use structured logging (via logger utility) with appropriate severity levels (`error`, `warn`, `info`, `http`, `debug`).

NFR-OBS-003 — Logs shall include correlation IDs to trace requests across services.

NFR-OBS-004 — Metrics shall be available for: request volume, failure rates, callback retries, and duplicate transaction detections.

NFR-OBS-005 — Alerting shall trigger on scheduler/callback/integration failures.

9.5 Maintainability

NFR-MAINT-001 — Frontend code shall pass ESLint with `@typescript-eslint`, `eslint-plugin-react-hooks`, and `eslint-plugin-react-refresh` rules.

NFR-MAINT-002 — Contract tests shall validate 1LINK payload and response schemas.

NFR-MAINT-003 — CI gates shall enforce lint, test, and build before merging.

NFR-MAINT-004 — Empty `interface` declarations shall use `type` aliases (enforced by `@typescript-eslint/no-empty-object-type`).

NFR-MAINT-005 — Tailwind plugins shall use ESM imports, not `require()`.

9.6 Usability

NFR-USE-001 — All key workflows shall be completable without referring to external documentation.

NFR-USE-002 — Empty states shall display meaningful messages with contextual action prompts.

NFR-USE-003 — All financial amounts shall be displayed in PKR with consistent formatting (`RsX,XXX,XXX`).

10. Security Requirements

SR-001 — All API traffic in production shall use HTTPS only. The server shall enforce

`REQUIRE_HTTPS=true` in production environments.

SR-002 — ETEA payment-controller API calls shall require API key authentication (`X-API-Key` header).

SR-003 — Callback and privileged endpoints shall enforce source IP whitelist validation.

SR-004 — The system shall validate webhook signatures (`X-Webhook-Signature`) for all 1BILL callbacks. Signature validation is enabled by default and controlled by

`REQUIRE_WEBHOOK_SIGNATURE` env var.

SR-005 — The system shall enforce callback idempotency using `X-Idempotency-Key` to detect and reject duplicate callback replays.

SR-006 — RBAC shall be enforced at the API layer with least-privilege access: users may only access resources matching their `role` and `tenantId` claims.

SR-007 — All create/update/delete and payment-critical operations shall produce auditable log entries.

SR-008 — Application logs and UI displays shall never expose plaintext passwords, JWT secrets, CNIC values, or API keys.

SR-009 — JWT secrets shall be strong random values in production. The server shall refuse to start if `JWT_SECRET` or `JWT_REFRESH_SECRET` is set to known insecure defaults (`change-me-in-production` , etc.).

SR-010 — Database credentials shall be set via environment variables. The server shall refuse to start if `DB_USER` or `DB_PASSWORD` is unset in production.

SR-011 — HTTP security headers shall be enforced via `helmet` middleware.

SR-012 — HTTP Parameter Pollution (HPP) attacks shall be mitigated via `hpp` middleware.

SR-013 — Rate limiting shall apply to all `/api/` routes (100 req/15 min general; 20 req/15 min for auth; 200 req/1 min for 1LINK gateway endpoints).

SR-014 — In production, the app shall trust exactly one reverse-proxy hop (`trust proxy: 1`) to correctly resolve client IP from `X-Forwarded-For` without allowing IP spoofing.

SR-015 — CORS shall restrict origins to the configured `CORS_ORIGIN` environment variable.

11. Constraints and Known Limitations

11.1 Current Implementation Gaps

ID	Area	Description	Impact
GAP-001	Routing	6 sidebar nav items link to unimplemented routes (Settings ×3, Roll Assignment, Admit Cards, Results) — results in 404	UX broken for those features
GAP-002	Persistence	Frontend currently uses in-memory mock data; page refresh loses all changes	Demo/dev only
GAP-003	Testing	Only a single stub test exists; no coverage for auth, components, routes, or API	Quality risk
GAP-004	ETEA data binding	<code>ETAIInvoices.tsx</code> reuses school invoice data instead of ETEA-specific records	Data accuracy issue
GAP-005	PDF export	"Export to PDF" triggers browser print dialog, not actual PDF generation	Feature incomplete
GAP-006	SMS reminders	Defaulter SMS reminder button exists in UI but no SMS integration is implemented	Feature incomplete
GAP-007	Billing policy	Auto/manual/hybrid invoice generation policy not yet implemented	Feature incomplete
GAP-008	SPA 404 redirect	The 404 page "Return to Home" uses <code>href="/"</code> causing full page reload that clears in-memory auth state	Auth state loss
GAP-009	Error handling	Most UI operations assume success; no error boundaries for failed API calls beyond top-level <code>ErrorBoundary</code>	UX degradation on errors
GAP-010	Reconciliation	Full payment reconciliation chain (invoice → ledger → transaction → audit) requires end-to-end verification	Payment integrity risk

11.2 Design Constraints

- The platform does **not** own ETEA applicant master data; ETEA remains the source of record.
- Custom bank rails outside 1LINK/1BILL contract are out of scope.
- Advanced BI/data lake features are deferred post-MVP.
- Consumer numbers must not exceed 1LINK maximum length constraints.

11.3 Technology Constraints

- Backend is Node.js (CommonJS modules); not ES modules — use `require()` not `import` in server code.
 - Frontend ESLint enforces no-empty-object-type and no-require-imports rules.
 - Lucide React named imports must be verified against the installed package version before use.
 - `npm run lint` runs workspace-wide; use `npx eslint <files>` for targeted checks.
-

Appendix A — Route Inventory

A.1 Frontend Routes (App.tsx)

Path	Component	Role Guard	Status
/	Redirect → /login	None	✓
/login	LoginPage	None (public)	✓
/admin	AdminDashboard	admin	✓
/admin/billers	BillerManagement	admin	✓
/admin/users	UserManagement	admin	✓
/admin/transactions	TransactionList	admin	✓
/admin/cashflow	CashFlow	admin	⚠ Stub
/admin/reports	Reports	admin	⚠ Stub
/admin/audit	AuditTrail	admin	✓
/admin/settings	(missing)	admin	✗ 404
/school	SchoolDashboard	school	✓
/school/students	StudentList	school	✓
/school/fee-plans	FeePlans	school	✓
/school/fee-ledger	FeeLedger	school	✓
/school/scholarships	Scholarships	school	✓
/school/billing	InvoiceList	school	✓
/school/invoices	InvoiceList	school	✓ (alias)
/school/defaulters	Defaulters	school	✓
/school/payments	SchoolPayments	school	⚠ Stub
/school/payment-programs	PaymentPrograms	school	✓
/school/reports	SchoolReports	school	⚠ Stub
/school/settings	(missing)	school	✗ 404

Path	Component	Role Guard	Status
/eta	ETADashboard	org/eta	✓
/eta/postings	ETEAPostings	org/eta	✓
/eta/applicants	ApplicantList	org/eta	✓
/eta/invoices	ETAINvoices	org/eta	✓
/eta/payments	ETAPayments	org/eta	⚠ Stub
/eta/reports	ETAREports	org/eta	⚠ Stub
/eta/services	(unrouted)	org/eta	✗ 404
/eta/payment-programs	(unrouted)	org/eta	✗ 404
/eta/roll-assignment	(unrouted)	org/eta	✗ 404
/eta/admit-cards	(unrouted)	org/eta	✗ 404
/eta/results	(unrouted)	org/eta	✗ 404
/eta/settings	(unrouted)	org/eta	✗ 404
*	NotFound	None	✓

Appendix B — API Endpoint Inventory

B.1 Backend Routes (server/src/index.js)

Route Prefix	Module	Purpose
POST /api/auth/login	auth	User authentication
POST /api/auth/refresh	auth	Token refresh
GET/POST/PUT /api/users	users	User management
GET/POST/PUT /api/tenants	tenants	Tenant management
GET/POST/PUT/DELETE /api/students	students	Student CRUD
GET/POST/PUT /api/invoices	invoices	Invoice management
GET/POST /api/billing	billing	Billing operations
GET/POST/PUT /api/applicants	applicants	Applicant management
GET/POST /api/org	org	Org/ETEA operations
POST /api/payment/callback	orgPayments	1BILL inbound callback
GET/POST /api/transactions	transactions	Transaction log
GET/PUT /api/settings	settings	Platform settings
GET/POST /api/notifications	notifications	Notification management
GET /api/audit-logs	audit	Audit log read
GET /api/reports	reports	Reporting endpoints
POST /api/1link/*	onelink	1LINK BillInquiry + BillPayment
POST /api/fetchbundle	fetchbundle	1LINK Fetch Bundle
GET/POST /api/bundles	bundles	Bundle management
POST /api/saas-gateway/*	saasGateway	SaaS gateway proxy
GET /api/admin-tools/*	adminTools	Admin sandbox tools
GET /api/health	(inline)	Health check

End of SRS — Billing Brilliance v1.0